Microcontrollers

Rex Joseph
Department of Electrical and
Electronics Engineering

Assembly Language Programming

Blink LED on P1.0 (IAR) (Yes differences in directives etc between IAR and CCS)

```
;-------------------------------------------------------------------------------
;  MSP430FR41xx Demo - Software Toggle P1.0
;
;  Description; Toggle P1.0 by xor'ing P1.0 inside of a software loop.
;  ACLK = n/a, MCLK = SMCLK = default DCO
;
;               MSP430FR41xx
;            -----------------
;        /|\|              XIN|-
;         | |               |
;         --|RST        XOUT|-
;           |               |
;           |           P1.0|-->LED
;
;  Texas Instruments, Inc
;  November 2014
;-------------------------------------------------------------------------------
#include "msp430.h"
;-------------------------------------------------------------------------------
        ORG     0F000h                  ; Progam Start (4K Flash device)
;-------------------------------------------------------------------------------
RESET       mov.w   #02200h,SP          ; Set stackpointer (512B RAM device)
StopWDT     mov.w   #WDTPW+WDTHOLD,&WDTCTL  ; Stop watchdog timer
SetupP1     bis.b   #001h,&P1DIR        ; Set P1.0 to output direction
                                        ;
Mainloop    xor.b   #001h,&P1OUT        ; Toggle P1.0
Wait        mov.w   #050000,R15         ; Delay to R15
L1          dec.w   R15                 ; Decrement R15
            jnz     L1                  ; Delay over?
            jmp     Mainloop            ; Again
            nop                         ;
                                        ;
;-------------------------------------------------------------------------------
;           Interrupt Vectors
;-------------------------------------------------------------------------------
            ORG     0FFFEh              ; MSP430 RESET Vector
            DW      RESET               ;
            END
```

Assembly Language Programming

```
;-------------------------------------------------------------------------------
;  MSP430FR41xx Demo - Software Toggle P1.0
;
;  Description; Toggle P1.0 by xor'ing P1.0 inside of a software loop.
;  ACLK = n/a, MCLK = SMCLK = default DCO
;
;            MSP430FR41xx
;          -----------------
;      /|\|              XIN|-
;       | |               |
;       --|RST        XOUT|-
;         |               |
;         |           P1.0|-->LED
;
;  Texas Instruments, Inc
;  November 2014
;
;-------------------------------------------------------------------------------
```

The story

Assembly Language Programming

```
#include "msp430.h"
;-------------------------------------------------------------------------------
        ORG    0F000h              ; Progam Start (4K Flash device)
;-------------------------------------------------------------------------------
RESET           mov.w   #02200h,SP                      ; Set stackpointer (512B RAM device)
StopWDT         mov.w   #WDTPW+WDTHOLD,&WDTCTL          ; Stop watchdog timer
SetupP1         bis.b   #001h,&P1DIR                    ; Set P1.0 to output direction
                        ;
Mainloop        xor.b   #001h,&P1OUT                    ; Toggle P1.0
Wait            mov.w   #050000,R15                     ; Delay to R15
L1              dec.w   R15                             ; Decrement R15
                jnz     L1                              ; Delay over?
                jmp     Mainloop                        ; Again
                nop                                     ;
                                                        ;
;-------------------------------------------------------------------------------
;       Interrupt Vectors
;-------------------------------------------------------------------------------
        ORG     0FFFEh                                  ; MSP430 RESET Vector
        DW      RESET                                   ;
        END
```

And the substance!

Assembly Language Programming

```
;-------------------------------------------------------------------------------
#include  <msp430xG46x.h>
;-------------------------------------------------------------------------------
            RSEG    CSTACK                  ; Define stack segment
;-------------------------------------------------------------------------------
            RSEG    CODE                    ; Assemble to Flash memory
;-------------------------------------------------------------------------------
RESET       mov.w   #SFE(CSTACK),SP         ; Initialize stackpointer
StopWDT     mov.w   #WDTPW+WDTHOLD,&WDTCTL  ; Stop WDT
SetupP5     bis.b   #001h,&P1DIR            ; P1.0 output
                                            ;
Mainloop    xor.b   #001h,&P1OUT            ; Toggle P1.0
Wait        mov.w   #050000,R15             ; Delay to R15
L1          dec.w   R15                     ; Decrement R15
            jnz     L1                      ; Delay over?
            jmp     Mainloop                ; Again
                                            ;
;-------------------------------------------------------------------------------
            COMMON  INTVEC                  ; Interrupt Vectors
;-------------------------------------------------------------------------------
            ORG     RESET_VECTOR            ; MSP430 RESET Vector
            DW      RESET                   ;
            END
;-------------------------------------------------------------------------------
```

Assembly Language Programming

Task is to write an assembly program that will repeatedly blink the LED1 on the MSP 430 Launchpad board every second, i.e., the LED1 will be on and off for about 0.5 seconds each.

Assembly Language Programming

Task is to write an assembly program that will repeatedly blink the LED1 on the MSP 430 Launchpad board every second, i.e., the LED1 will be on and off for about 0.5 seconds each.

Solution:

Step 1: Analyze the problem statement.

First study schematics of the board. Locate LED1 output port. What microcontroller port pins are connected to LED1? How the LED1 port is actually connected to a physical led (a diode through a resistor).

Step 2. Develop a plan.

If we want LED1 on, we should provide a logical '1' at the output port of the microcontroller (port P1.0), and a logical '0,' if we want it off.

We could take several approaches to solving this problem. The simplest one is to **toggle the port P1.0** and have **0.5 seconds delay in software**.

Assembly Language Programming

After initializing the microcontroller, the program will spend all its time in an infinite loop (LED1 should be repeatedly blinked).

Inside a loop we will toggle the port P1.0 and then wait for approximately 0.5 seconds.

The port P1.0 toggling can be done using an XOR operation of the current value of the port (P1OUT) and the constant 0x01, i.e., (P1OUT=P1OUT xor 0x01).

Software delay of 0.5 seconds can be implemented using an empty loop with a certain number of iterations.

We need to know number of instructions, the cycles and clock frequency

Assembly Language Programming

The comments in a single line start with a column character (;).
Multi-line comments can use C-style /* comment */ notation.

#include ; This is a C-style **pre-processor directive** that specifies a header file to be included in the source.
The header file includes all **macro definitions**, for example, special function register addresses (WDTCTL), and control bits (WDTPW+WDTHOLD).

RSEG is a **segment control assembler directive** that controls how code and data are located in memory.
RSEG is used to mark the beginning of a relocatable **code or data segment**.
CODE and DATA are recognized segment types that are resolved by the linker.

Assembly Language Programming

The IAR XLINK linker can recognize any other type of segment (e.g., CSTACK for code stack).

Upon powering-up the MSP430 control logic always generates a RESET interrupt request (it is the highest priority interrupt request).

The value stored at the address 0xFFFE (the last word in the 64KB address space) is reserved to keep the starting address of the reset handler (interrupt service routine), and the first thing that the microcontroller does is to fetch the content from this address and put it in the program counter (PC).

Thus, the starting address of our program should be stored at location 0xFFFE. The linker is responsible for this step.

Assembly Language Programming

The symbolic name RESET (the starting address of our program) is used to load the reset vector in the interrupt vector table.

First instruction initializes the stack pointer register.

The move instruction at the StopWDT label, sets certain control bits of the watchdog timer to disable it.
The watchdog timer by **default will be active** generating interrupt requests periodically.
As this functionality is not needed in our program, we simply need to disable it.

Parallel ports in MSP430 microcontroller can be configured as input or output. Control register PxDIR determines whether the port x is input or output (we can configure each individual port pin).

Assembly Language Programming

Our program drives the port pin P1.0, so it should be configured as the output. The LSB bit of the P1DIR is set to 1, while other bits are unchanged (**0 by default**).

The main loop starts at Mainloop **label**. The xor instruction performs toggling operation

The code starting at label Wait implements software delay.
To exactly calculate the software delay we need to know instruction execution time and the clock cycle time. The register R15 is loaded with 50,000. The dec.w instruction takes 1 clock cycle to execute, and jnz L1 takes 2 clock cycles to execute (*this can be determined by using the Simulator and the Register View CYCLECOUNTER field*).

Assembly Language Programming

The total time per iteration is **3 clock cycles**.

Determining clock cycle time requires in-depth understanding of the FLL-Clock module of the MSP430.
The processor clock frequency is approximately 1MHz for our configuration.
The total delay is thus 50,000*3*1us=150,000us.
Obviously, to meet our requirements we need to increase the number of instructions in the software delay loop (to be 10).
We can do it by adding **nop** instructions

The final program code that satisfies the requirements is shown

Assembly Language Programming

```
#include  <msp430xG46x.h>
;-------------------------------------------------------------------------
            RSEG    CSTACK                  ; Define stack segment
;-------------------------------------------------------------------------
            RSEG    CODE                    ; Assemble to Flash memory
;-------------------------------------------------------------------------
RESET       mov.w   #SFE(CSTACK),SP         ; Initialize stackpointer
StopWDT     mov.w   #WDTPW+WDTHOLD,&WDTCTL  ; Stop WDT
SetupP5     bis.b   #001h,&P1DIR            ; P1.0 output
                                            ;
Mainloop    xor.b   #001h,&P1OUT            ; Toggle P1.0
Wait        mov.w   #050000,R15             ; Delay to R15
L1          dec.w   R15                     ; Decrement R15
            nop
            nop
            nop
            nop
            nop
            nop
            nop
            jnz     L1                      ; Delay over?
            jmp     Mainloop                ; Again
                                            ;
;-------------------------------------------------------------------------
            COMMON  INTVEC                  ; Interrupt Vectors
;-------------------------------------------------------------------------
            ORG     RESET_VECTOR            ; MSP430 RESET Vector
            DW      RESET                   ;
            END
```

THANK YOU

Rex Joseph

Department of Electrical and Electronics Engineering

rexjoseph@pes.edu

+91 80 6666 3333 Extn 515